SYMBOL / CATAPULT

AtomicBundle

既存署名済み注文を包む限定拡張

設計書・共通仕様・Codex実装指示書

実装前の提案仕様

既存署名済みAggregate注文を保持し、funding・注文実行・sweepを外側の認可に束縛するDEX限定仕様。

基準	内容
対象ソース	symbol/symbol@6c04695555230c89a6ffa2ac4d41774dcac77ae3
用途	同一プラグインを読み込む3ノードの隔離private-net PoC
版	0.1 / 2026-10-25
収録	対象設計書、共通仕様、Codex共通指示、対象Codex指示を全文収録
正本	同梱Markdown。署名fixtureと生成スクリプトはキット内の独立ファイル。

このPDFは実装済みコード、実行済み試験結果、安全性監査の証明ではありません。未実行の項目はNOT RUNとして扱います。図は設計本文の関係を可視化したものです。

目次

AtomicBundle v1 設計書 - 既存署名済み注文を使う DEX 限定 PoC 3

文書の位置付け 3

1. 何を保証するか 4

2. 登場主体と必須の不変条件 4

3. wire 形式と canonical root 5

4. 外側署名・連署・既存注文の検証 6

5. 残高の意味と P の dust 7

6. native 検証経路と実行順序 7

7. 通常 Aggregate と共有する消費状態 8

8. PoC 仮設定と実装配置 9

9. 受入試験 9

10. 完了条件と残る範囲 11

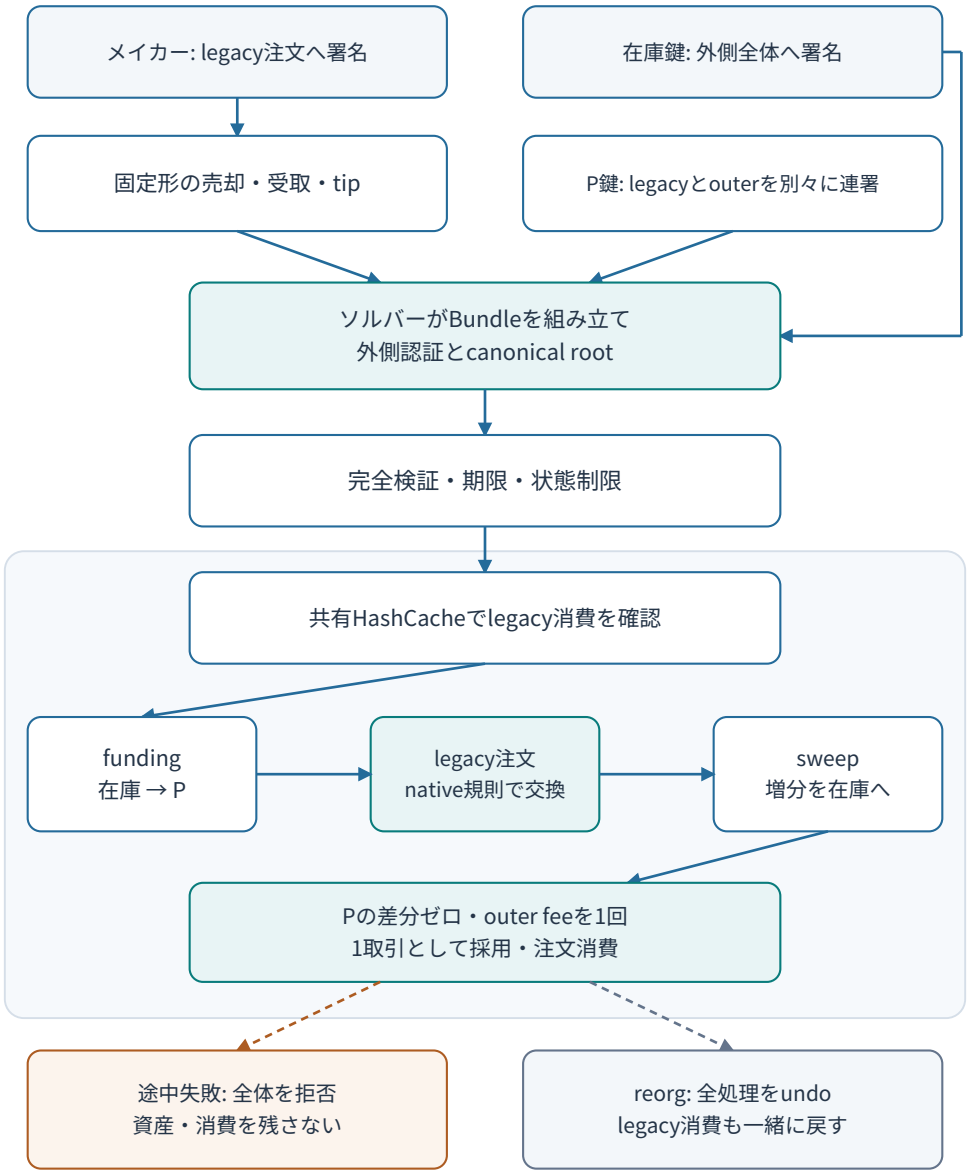
一次資料 11

両PoCの共通仕様と検証条件 12

Codex 共通作業指示 15

Codex 実装指示書 - AtomicBundle v1 17

AtomicBundle v1 設計書 - 既存署名済み注文を使う DEX 限定 PoC



funding / sweepは単独で有効な署名済みTransferにしない

受信した無効blockは全体拒否 / 候補作成時は外側1件をundo

図版: 設計本文に記載した処理と認可の関係。実装・実行結果を示すものではありません。

文書の位置付け

これは実装・実行・安全性実証をまだ行っていない設計仕様である。対象は、新規 genesis から起動する隔離された private chain のみ。公開ネットワークへの接続、実資産の投入、mainnet の更新・移行は対象外とする。共通条件は 共通仕様、実装手順は Codex 指示書 を参照する。

対象	固定する版
Symbol / catapult の基準	symbol/symbol@6c04695555230c89a6ffa2ac4d41774dcac77ae3

対象	固定する版
既存注文形式の参照	HinaBlogChannel/heika@891a2cb17a0d52c85ed85594d8283c844481a6cb の examples/intent-dex
新機能	AtomicBundle、wire version 1、DEX 用の固定形のみ

以下の新しい型名・設定名は設計上の提案であり、既存 Symbol API ではない。公開 EntityType の数値は割り当てない。実装の M0 で既存型・もう一方の PoC と衝突しない実験用値を決め、schema とテストの一箇所に定義する。

図の編集用: Mermaidソース / SVG。細かい成立条件とバイト形式は以下の本文を正本とする。

1. 何を保証するか

一つの top-level transaction に、次の三つをこの順番で入れる。

1. 在庫アカウントから P への funding。
2. メイカーが署名済みの、既存形式の AggregateComplete V3 注文。
3. P から在庫アカウントへの sweep。

funding と sweep は単独の通常 Transfer として署名しない。 外側の署名が認可する埋め込み操作にする。独立して有効な Transfer を三つ包むだけでは、資金投入だけを切り出した競合ブロックを防げない。[S1][S2]

新しいプロトコル規則は、三つの処理と注文消費を一つの取引として検証・反映する。ブロックが別の枝に置き換わっても、別枝で再実行できるのは正しい Bundle 全体であり、在庫提供者の承認を funding だけに流用できない。

ハーベスターがソルバーである必要はない。更新済み private chain の任意のハーベスターが通常の選択処理で Bundle を収録できる。これは新たな取引規則の追加であり、現行 mainnet に一台だけプラグインを追加して成立する方式ではない。[S3]

v1 の制限

注文は一つ、論理子操作は三つ、注文内の embedded Transfer は三つに固定する。Aggregate Bonded、任意の再帰、部分約定、複数注文、alias、メッセージ、マルチシグ、任意の管理操作は拒否する。maker・inventory・P は相異なる通常アカウントとし、単一鍵の認証を固定する。未対応型を黙って読み飛ばす動作は認めない。

2. 登場主体と必須の不変条件

主体	認可する内容	保持する鍵
maker	既存 Aggregate の売却・受取・tip	maker の鍵
inventory / solver	Bundle 全体、funding、外側ネットワーク手数料	在庫の鍵
P	既存注文の P 側、Bundle の sweep	公開された P の鍵
harvester	通常のブロック署名	別のハーベスト鍵

maker は外側 Bundle に署名しない。solver は maker の署名を変更しない。既存注文に対する P の連署と、外側 Bundle に対する P の連署は別の署名である。前者だけで sweep を承認したことにはしない。

必須不変条件は次のとおり。

- inventory の資産を動かせるのは、その inventory が外側全体を署名した場合だけ。
- P の sweep は、外側ハッシュへの P 連署を必要とする。

- maker の資産移動は、署名済み注文と完全に一致する。
- 正常終了時、全モザイクについて P_after = P_before。
- 失敗した Bundle の残高変更・注文消費・receipt の変更は残らない。
- 一つの既存注文は、通常 Aggregate と Bundle の経路を合わせて一回だけ使える。
- 同じ入力・同じ親状態に対する状態ハッシュと receipt hash は全ノードで一致する。

P が公開鍵アカウントであることをノードが秘密鍵の公開状況から判定する必要はない。ノードは二つの必要な P 署名を検証する。P の制限やマルチシグ化によって認証条件を満たさなくなった場合、Bundle 全体を拒否する。

3. wire 形式と canonical root

3.1 外側の構成

共通 Transaction header は基準版の既存形式を使う。続く body は次の提案で固定する。整数は unsigned little-endian、ハッシュと鍵は raw bytes とする。

順序	フィールド	大きさ	条件
共通	既存 Transaction header	128 bytes	version 1、新型、同一 network、通常の signature / MaxFee / Deadline
1	ActionsRoot	32 bytes	下記の計算結果
2	ActionsSize	u32	ActionStream 全体の bytes 数
3	ActionCount	u16	必ず 3
4	OuterCosignaturesCount	u16	必ず 1
5	BodyReserved	8 bytes	全て 0
6	ActionStream	可変	三つの frame を連結
7	P の outer cosignature	104 bytes	基準版の Cosignature 形式、version 0、P の公開鍵

外側の Size は厳密に 128 + 48 + ActionsSize + 104 と等しいこと。余剰 bytes、欠損、整数 overflow、非ゼロ reserved、別解釈できる encoding を拒否する。外側は top-level のみで、他の Aggregate / Bundle に埋め込めない。

3.2 子操作 frame

frame の header は 16 bytes とする。

```
LE16(kind) || LE16(version=1) || LE16(flags=0) || LE16(reserved=0)
|| LE32(index) || LE32(payloadLength) || payload || zeroPadding
```

zeroPadding は 16 + payloadLength を 8 の倍数にする 0 bytes、長さは 0〜7。payloadLength は padding を含まない。frame 全長は header と padding を含む。index は順番どおり 0、1、2。kind はこのコンテナ内部の値であり、公開 transaction EntityType ではない。

index / kind	payload	必須条件
0 / 1	native EmbeddedTransferTransaction V1 の bytes	inventory → P、get の単一モザイク、message 空

index / kind	payload	必須条件
1 / 2	完全な AggregateComplete V3 の bytes	maker の主署名、P の既存連署一つ、Transfer 三つ
2 / 3	native EmbeddedTransferTransaction V1 の bytes	P → inventory、give と tip の固定増分、message 空

funding / sweep の payload に、通常取引用の署名・MaxFee・Deadline は存在しない。通常 Transfer のバイト列を payload として受理しない。native embedded codec の size・予約領域・モザイク順序も検証し、parse 後の canonical 再 serialize が入力と一致することを fixture で確認する。

3.3 root の厳密な規則

```
D = ASCII("SYMBOL_ATOMIC_BUNDLE_ACTIONS_V1")  // NUL・改行を付けない
ActionStream = frame[0] || frame[1] || frame[2]
ActionsSize = byte_length(ActionStream)
ActionsRoot = SHA3-256(D || LE32(3) || LE32(ActionsSize) || ActionStream)
```

これは可変の Merkle tree ではなく、型付きの順序列全体への一つの commitment である。種類・順序・個数・長さ・padding・子 bytes を固定する。外側 body の ActionCount、ActionsSize と root 入力が一致しなければ拒否する。legacy payload 内の P 連署は root に含む。外側 P 連署は root に含めない。

JSON は人間向け入出力にだけ使い、署名や root の入力にはしない。opaque bytes を使う root fixture は「container codec 専用、実行可能な取引ではない」と明記し、有効な native payload の fixture と区別する。

4. 外側署名・連署・既存注文の検証

4.1 外側認証

inventory の主署名は、基準版の取引署名規則と generation hash seed を使う。新プラグインの dataBuffer は共通 header の署名対象フィールドから ActionsRoot・ActionsSize・ActionCount・OuterCosignaturesCount・BodyReserved までを含め、ActionStream と末尾の outer cosignature は含めない。ActionStream は root を通して拘束される。[S2]

外側 MaxFee、Deadline、network、type、version の変更で主署名が無効になること。共通 header の signer と署名検証鍵の関係も既存規則どおりにする。独自のハッシュ式や「署名対象 header 全体」という曖昧な実装を作らず、基準版の EntityHasher / signature publisher と byte vector で一致を確認する。

P の outer cosignature は canonical な外側 transaction hashに署名する。外側ハッシュは outer cosignature を除外して計算し、連署の前後で変わらないこと。generation seed の追加は基準版の Aggregate 連署と同じ扱いにし、二重に付加しない。merkleSupplementaryBuffers は P の公開鍵 32 bytes を一つの buffer として返す。連署の signature bytes はこの buffer に入れない。native Aggregate と同じ構成を基準版のコードと vector で確認する。[S2]

4.2 既存注文の固定形

legacy Aggregate は以下の順序を厳密に要求する。

内部順序	signer → recipient	内容
1	maker → P	give asset を give amount
2	P → maker	get asset を get amount
3	maker → P	network currency を tip amount

各 Transfer は一モザイク、空メッセージ、数量 > 0。give asset と get asset は異なる。address と mosaic ID は literal のみで、alias 用表現を拒否する。currency ID は private chain の設定値を使い、mainnet の値を埋め込まない。P は受取先・内部 signer・既存連署 signer・外側連署 signer・sweep signer の全箇所で一致させる。

次を全て検証する：full size、共通 reserved、AggregateComplete の型、version が厳密に 3、network、generation seed に対する maker 主署名、native TransactionsHash、内部 payload size / padding、Transfer の型と version、必要連署の完全性、余分な連署の不在、P 既存連署の署名、native deadline と最大 lifetime、既存注文ハッシュの未消費、通常の残高・口座状態・モザイク状態・送受信制限。[S1][S2][S4]

outer Deadline は legacy Deadline 以下とし、両方を実際の block time で検証する。outer と legacy の期限を同じ値に上書きしてはならない。legacy の signed MaxFee も原本のまま保持する。

5. 残高の意味と P の dust

give を (A,a)、get を (B,b)、currency を C、tip を τ とする。全て最小単位の整数であり、小数や浮動小数点は使わない。

```
funding: inventory - bB, P + bB
order:   maker - aA + bB - τC, P + aA - bB + τC
sweep:   P - aA - τC, inventory + aA + τC
```

したがって、P の変化はゼロ、maker の変化は署名済み注文どおり、inventory の変化はその逆になる。A=C の場合、sweep の数量は checked addition で a+τ にまとめる。別モザイクなら canonical な ID 順に並べる。B=C の場合も同じベクトル式で処理できる。overflow、数量 0、重複 ID を別要素で表す形式を拒否する。

P の初期残高ゼロは要求しない。 sweep は当該注文で増えた aA+τC だけであり、sweep-all は禁止する。既存 dust は全て保持し、無関係なモザイクを含め P_after=P_before を確認する。これにより、第三者の dust 送金だけで注文を無効にする条件を増やさない。P の残高を他人が独立取引で動かせることと、本 Bundle が在庫資金を失うことは区別する。

手数料の新規則

- ネットワーク手数料は outer fee 一回だけ。solver / inventory が払う。
- 実額は native の outer serialized size × block fee multiplier。outer MaxFee 以下であること。
- legacy child の actual network fee は 0。signed MaxFee は変更せず、0 がその上限内であることを扱う。
- funding / sweep に別のネットワーク手数料はない。tip は全量 inventory へ回収し、そこから sweep fee を引かない。
- tip とネットワーク手数料は別物。currency 保存を、maker・inventory・P・harvester・beneficiary の収支と既存 fee 分配規則で確認する。

これは既存 Aggregate を通常どおり一件課金してから「払い戻す」実装ではない。子に native publisher を無条件適用すると、旧 MaxFee の仮徴収や二重課金が起き得る。[S4]

共通仕様に従い、private chain では TotalVotingBalanceCalculationFix=Height(1) 相当の実在設定を M0 で確認し、activation height 2 以降で custom 処理後に outer fee debit を行う。funding 全量はその実行時に必要で、sweep 後の currency 残高が outer fee を賄えること。custom 前に funding と fee の合計を無条件に要求する独自 gate は置かない。UT の MaxFee 仮徴収と実ブロックの actual fee を分けて試験する。

6. native 検証経路と実行順序

6.1 childPlugin.publish() だけでは不足する

基準版では、custom plugin publisher と、共通の public-key / entity / transaction / deadline / fee / main-signature 通知を作る処理が分かれている。Aggregate plugin だけを呼ぶと、maker の共通認証・期限・重複検証が欠ける。[S2][S4]

新たに限定 legacy-order adapterを実装する。これは既存 API 名ではなく、必要な実装責務を示す名称である。adapter は少なくとも maker の AccountPublicKey、Entity、TransactionNotification、TransactionDeadline、actual fee 0 の TransactionFee、replay protection を有効にした maker の主署名検証を維持し、そのうえで既存 Aggregate 固有の通知と連署検証を通す。子の network fee debit / receipt は発生させない。TransactionNotification には maker、legacy の型、legacy hash、legacy Deadline を使う。

資金投入・回収についても Transfer plugin の custom 通知だけに依存しない。inventory と P の外側認証、native Transfer の構造と送受信通知、account / mosaic restrictions を確認する。v1 の追加制約として、全五つの資産移動の signer に Transfer の outgoing operation permission を要求する。inventory は funding、maker は give と tip、P は get と sweep が対象である。top-level の TransactionNotification は inventory の新型操作、子の TransactionNotification は maker の Aggregate 操作を検査するが、各 embedded Transfer の操作許可を自動的に検査するとは仮定しない。

この追加条件は Bundle 内だけに適用し、通常 Aggregate の既存ルールを変更しない。maker・inventory・P の非マルチシグ条件と、remote account 等の既存認証制約も検証する。native helper を再利用できない場合は専用 validator で同じ許可条件を明文化し、未対応を安全側に拒否する。

6.2 論理処理

1. 全体の size / canonical root / 固定形 / 四署名を確認する。
2. 外側と子の期限、権限、未消費、静的な制限を検証する。
3. native の一時 cache delta 上で funding を反映する。
4. legacy の共通通知・Aggregate 処理を通し、その時点の残高と制限で注文を実行する。
5. native の同じ delta 上で固定増分の sweep を実行する。
6. P の変化ゼロ、fee、通知 source、注文消費を含む結果を検証し、成功した取引だけを採用する。

後段失敗では前段の変更も取り消す。native の処理は通知ごとの validator / observer を使うため、単純な C++ 関数呼出しが database transaction になるとは仮定しない。UT 検証、ブロック候補生成、ブロック受入れ、rollback、再起動の全経路を試験する。[S5]

6.3 深さ・source の固定

論理子は 3、展開した内部 entity は funding 1 + legacy container 1 + legacy embedded 3 + sweep 1 = 6。新 plugin の embeddedCount() は 6、native DeepCount は外側を含め 7 とする。基準版のブロック集計は自動再帰せず 1 + embeddedCount を加算するため、この値を cap と統計に反映する。[S6]

receipt source の primary は外側の top-level transaction index のままにする。secondary は funding=1、legacy container=2、legacy 内部=3/4/5、sweep=6 と固定し、外側へ戻る際は 0 に戻す。legacy publisher の相対 +1 を adapter でこの範囲に閉じる。子に full top-level publisher を使って primary を進めてはならない。v1 は新しい独自 receipt 型を追加せず、既存 receipt と source の処理を維持する。source trace と receipt hash の両方を検証する。[S2][S4][S7]

7. 通常 Aggregate と共有する消費状態

legacy order の identity は、基準版と同じ方法で計算する canonical legacy transaction hash。outer hash や ActionRoot を注文識別子にしてはならない。P の連署を含めた wrapper の hash で代用してもならない。

既存 HashCache のキー TimestampedHash(legacy Deadline, legacy hash) を、通常 Aggregate 経路と Bundle 経路で共用する。既存 UniqueTransactionHash validator と TransactionHash observer は TransactionNotification を使ってこのキーを検索・挿入し、rollback 時は除去する。adapter が同じ通知を発行することで、両経路の消費を一つにする。[S8][S9]

同一ブロック内でも最初の消費が次の検証から見えること。Bundle A と Bundle B の outer hash が異なっても、同じ legacy 注文なら後者を拒否する。通常 Aggregate の後の Bundle、その逆、通常→通常、Bundle→Bundle を全て試験する。

pruning は legacy Deadline と基準版の finalized-time / retention 規則に従い、短い outer Deadline で子の消費を早く消してはならない。rollback 可能範囲で再構築・再起動しても消費状態を保持する。AtomicBundle 専用の cache や新しい

Merkle state root は v1 では作らない。既存 HashCache はこの PoC のために Merkle root を追加する対象ではない。再起動後の cache 再構成と expiry 後の期限拒否を独立に確認する。[S8][S9]

8. PoC 仮設定と実装配置

以下は公開ネットワーク値ではなく、この隔離 PoC の提案値である。

項目	値 / 規則
maxAtomicBundleBytes	4,096 bytes、外側連署を含む
maxLegacyOrderBytes	2,048 bytes、既存 P 連署を含む
logicalActionCount	厳密に 3
legacyEmbeddedCount	厳密に 3
outerCosignaturesCount	厳密に 1、P のみ
legacyCosignaturesCount	厳密に 1、P のみ
signature verification count	厳密に 4：inventory / outer P / legacy maker / legacy P
maximum depth	outer → legacy → Transfer の固定形のみ、再帰禁止
block DeepCount contribution	7
maxOuterLifetime	2 時間、native の設定・許容範囲と照合
legacy lifetime	private chain の既存 Aggregate 設定を使う
outer Deadline	legacy Deadline 以下
native private fork heights	StrictAggregateTransactionHash、UniqueAggregateTransactionHash、TotalVotingBalanceCalculationFix を Height(1) 相当へ統一。実設定名・適用を M0 で確認
AtomicBundle activation	height 2。native Aggregate V3 の有効化とは別に検証

実装は C++ transaction plugin、model / notification、validator、legacy adapter、native hashcache の統合、schema / codec、unit と integration tests を含む。例として client/catapult/plugins/txes/atomic_bundle 以下を新設できるが、実在する CMake・登録・schema 生成規則を M0 で読んで決める。既存 plugin の共有処理を変更する場合は差分を小さくし、通常 Aggregate の既存試験を必須 gate にする。

成果物には三ノード用 manifest / 起動停止手順、独立 genesis、決定論的 test-only 鍵、genesis hash、設定 hash、ソース commit、生成 fixture、実行コマンド、結果ログを残す。genesis 生成も同じ native fork 設定で揃え、最初に単独 Aggregate V3 の受理を確認する。AtomicBundle の有効化だけで既存 V3 validator が有効になるとは仮定しない。REST の表示をコンセンサス検証に使わず、public seed と実ウォレットの設定を取り込まない。

9. 受入試験

全ての試験は単なる予想ではなく、実測結果と artifact を残す。比較表で未実行を成功に数えない。

ID	試験	必須の結果
AB-01	canonical codec / root vector	三種類の長さ、padding、順序、count、root が一致。異常値は拒否

ID	試験	必須の結果
AB-02	outer 認証	fee / deadline / recipient / amount / kind / root の一ビット改変を拒否。P 連署の欠損・流用・余分な連署も拒否
AB-03	legacy 完全検証	maker 主署名、P 連署、TransactionsHash、型・version・network・deadline・size の破損を全て拒否
AB-04	基本約定	三ノードで同じ Bundle を収録。maker / inventory の期待差分、P の差分ゼロ、同一 state / receipts / block hash
AB-05	非ゼロ fee	multiplier 0 と非ゼロの双方。outer 一回だけ課金、maker child fee 0、tip 全量、native 分配を含む総額一致
AB-06	P dust	give / get / currency / 無関係モザイクの dust を別々・同時に保持。sweep-all・増量を拒否
AB-07	残高・制限失敗	maker 不足、inventory 不足、restricted mosaic、口座送受信・操作制限、P のマルチシグ化を拒否し、全差分なし
AB-08	未対応形	Bonded、再帰、alias、partial、メッセージ、管理型、複数注文、余分な embedded を拒否
AB-09	通常→通常	既存ハッシュ保護を維持。同一・別ブロック、期限内の二重実行を拒否
AB-10	通常→Bundle	先に通常経路で消費した legacy 注文を Bundle へ入れても拒否
AB-11	Bundle→通常	Bundle が消費した注文を単独 Aggregate として再利用できない
AB-12	Bundle→Bundle	outer の signer / deadline / fee / hash が異なっても同じ注文を二重実行できない
AB-13	遅い段階の失敗	test harness 限定で legacy 適用後・sweep 中・outer fee 前に failure を注入し、funding・消費・fee・receipt が残らない。署名不正は AB-02/03 で別に確認
AB-14	旧方式の対照実験	旧 T1/T2/T3 方式だけで、有効な早い timestamp の競合ブロックに T1 を再収録できる条件を再現しログ化
AB-15	funding 抽出	新 funding payload を通常 Transfer として送信・直接収録しても在庫署名がなく拒否。元外側署名の流用も失敗
AB-16	fork / replay	未 finalized の同一親から競合枝を作成。全 Bundle 再収録なら全額一緒、除外なら全額なし。再署名せず改変した Bundle は拒否
AB-17	rollback / restart / prune	消費キーを旧枝で undo、新枝で再検証。未消費・期限内なら inventory と P が正しく再署名した別 wrapper は受理。restart 後も消費を保護し、outer expiry で子キーを早期削除しない
AB-18	source / DeepCount / limits	secondary 1〜6 と復帰 0、DeepCount 7、cap 境界、native receipt hash を全ノードで一致

AB-14 の競合ブロックは実際の VRF・hit・difficulty・score 条件を満たすものを使い、単に timestamp を書き換えた不正ブロックで代用しない。元ブロックが finalization 済みの場合の拒否も正常な対照として確認する。旧方式の弱点を private chain 外へ試してはならない。

AB-09〜12 は同一ブロックと別ブロックの両方で実施する。UT の事前検証で落ちる場合だけで済ませず、無理にブロックへ入れた入力も native validator が拒否することを確認する。重複を混ぜた不正ブロックはブロック全体を採用しない。

AB-13 のために本番の署名検証を遅らせない。署名不正が前段で拒否されたケースと、状態を一部変更した後に undo したケースを区別してログへ残す。故障注入 hook はテスト専用で、本番 plugin から有効にできないこと。

10. 完了条件と残る範囲

完了は、説明用 mock ではなく基準版 C++ plugin が build され、三ノードの実行ログ・残高・hash・rollback 証跡で上記条件を確認した状態をいう。P 連署や maker の共通検証を省いて「デモ成功」にしてはならない。失敗条件を報告する実装も必要であり、未解決の項目はブロッカーとして残す。

PoC 完了後も、mainnet 適用の安全性、公開型番号、複数注文の効率、部分約定、経済性、運用、外部監査は別課題である。この仕様はネットワーク更新の承認や本番展開を含まない。

一次資料

- S1: heika 既存注文・funding・sweep 実装
- S2: AggregateTransactionPlugin.cpp
- S3: Symbol Plugin 公式説明
- S4: NotificationPublisher.cpp
- S5: BatchEntityProcessor.cpp
- S6: BlockUtils.cpp
- S7: SourceChangeObserver.cpp
- S8: TransactionHashObserver.cpp
- S9: UniqueTransactionHashValidator.cpp
- S10: Aggregate の catbuffer schema
- S11: DeadlineValidator.cpp
- S12: HashCache.h

両PoCの共通仕様と検証条件

仕様版: 0.1 / 2026-10-25 / ステータス: 実装前のPoC仕様

1. 対象と結論

IntentSettlementは、メイカーの条件署名とソルバーの決済署名を検証する交換専用取引を追加します。AtomicBundleは、限定された既存署名済みAggregateと、その前後の内部操作を一体で実行する取引を追加します。

共通する目標は、ハーベスターの善意や取引の並べ方に依存せず、支払認可と資産移動の不可分性を検証規則で強制することです。外側の箱を作るだけでは、独立に有効な通常Transferの署名を失効させられません。

2案は独立した比較対象です。IntentのコードをAtomicへ自動的に取り込まず、共通化は両方の最小仕様が安定してから別変更で行います。

2. 固定した一次資料

資料	確認対象
symbol/symbol	6c04695555230c89a6ffa2ac4d41774dcac77ae3
heikaの元の交換例	891a2cb17a0d52c85ed85594d8283c844481a6cb
公式プラグイン説明	docs.symbol.dev / plugin.html
公式Aggregate説明	docs.symbol.dev / aggregate-transaction.html

以下のAPI名は固定コミットで確認した実在の拡張点です。新しい取引型名・通知名・キャッシュ名は各設計書の提案であり、既存APIではありません。

- PluginManager: 取引型、validator、observer、cacheの登録。
- TransactionPlugin: 署名対象、追加Merkleデータ、embedded件数、top-level可否。
- AggregateTransactionPlugin: Aggregateはembedded不可。子の固有通知と共通の取引処理は別。
- NotificationPublisher: 期限、手数料、通常署名、取引ハッシュ通知の共通経路。
- HarvestingUtFacadeFactory: 候補取引への適用と失敗時undo。
- UniqueTransactionHashValidator と TransactionHashObserver: 既存取引ハッシュの重複防止とrollback。
- CatapultCache: サブキャッシュrootから全体StateHashを計算。

3. ネットワーク境界

3.1 私設ネットワークのみ

公式C++ catapultを対象とし、同一バイナリ・同一設定の独立した3プロセス以上で検証します。RESTだけのモデルやメモリ上の独自台帳は補助資料です。プラグインのロード、ブロック受け入れ、状態更新、別プロセスへの伝播を確認して初めてネットワーク試験が成立します。

公開mainnet/testnetのseedノードを接続先に使いません。鍵と資金は使い捨ての私設ネットワーク専用とし、既存ウォレットの秘密鍵を入力しません。ネットワーク識別子だけで環境を区別せず、私設チェーン固有のgeneration hash seedで署名を分離します。

3.2 新しい状態rootとgenesis

新しいMerkle対応サブキャッシュは、空であってもStateHashの構成に影響し得ます。Intentの新キャッシュを、既存チェーンのスナップショットへ単に後付けしてはいけません。nemgen側と全検証ノード側が同じプラグインとキャッシュ構成を持つ、fresh private genesisを作ります。

PoCの検証有効化高さは 2 を初期案とします。これは全ノードが初めから新プラグインを持つ私設ネットワーク内の境界試験です。既存メインネットへの安全な状態移行や、旧ノードと共存できることを実証する設定ではありません。

旧バイナリへの負試験は、新しい取引型のデコード／検証拒否を確認します。IntentではgenesisのStateHash構成も異なり得るため、旧バイナリが新チェーンの最初のブロックから追従できることを前提にしません。

外側feeの通知順を固定する

基準ソースの NotificationPublisher は、fork設定によりCustom通知より前にfeeを引く場合と、後に引く場合があります。PoCでは取引型の有効化高さ2以降をCustom処理 → 外側fee debitへ固定します。ForkHeights.TotalVotingBalanceCalculationFix の私設値を Height(1) とする構成を初期案とし、実際の設定キー表記・受理・通知順をM0で確認してください。

古いpresetの大きなfork高さをそのまま流用すると、プラグインが初期残高として「支払額 + MaxFee」を検査する前にfeeが減り、二重にfee資金を要求する不整合が生じます。最大手数料とちょうど同額のfee資金を持つ境界ケースを必須試験にして、この順序を確認します。mainnetのfork設定を変更する指示ではありません。

既存Aggregate V3の有効化も揃える

AtomicBundle v1はLegacy AggregateComplete V3を検証します。新しいAtomicBundleだけを高さ2で有効化しても、既存Aggregateのversion validatorがV3を拒否する設定では動きません。私設ネットでは ForkHeights.StrictAggregateTransactionHash と ForkHeights.UniqueAggregateTransactionHash も Height(1) を初期案として設定し、ネイティブの単独V3とBundle内V3の両経路がテスト開始時から同じ版・hash規則を使うことを確認します。キー表記・genesis生成との整合・境界の受理はM0の必須確認です。既存mainnetのfork履歴を変更する意図はありません。

3.3 型番号とABI

取引型の数値、FacilityCode、キャッシュ登録番号をこの文書で既存番号へ仮割当しません。CodexはM0で対象コミットのenum・登録処理・予約値を確認し、衝突のないPoC専用値を protocol-abi.md に記録します。クライアント、catbuffer、ノードは同じ定義を参照します。

4. 共通の安全性条件

ID	必須条件
C-01	認可されていない第三者の残高を減らさない。outer署名で認可される口座と、条件署名で認可される口座を区別する。
C-02	交換の片側、資金投入だけ、注文消費だけを確定させない。
C-03	同じ注文を同一ブロック・後続ブロック・別の包み方から二重実行できない。
C-04	reorg時は残高、注文消費、必要なreceiptとcacheを一貫して元へ戻す。
C-05	ネットワーク、取引型、版、金額、宛先、期限、手数料条件を署名対象と整合させる。
C-06	マルチシグ、アカウント制限、モザイク制限・存続期間などを迂回しない。v1で非対応の状態は明示的に拒否する。
C-07	型付き・長さ付き・順序付きの正規バイト列を使う。JSON文字列表現を署名しない。
C-08	全整数計算に上限とoverflow検査を置く。金額にfloatを使わない。
C-09	外側の実サイズに基づくネットワーク手数料を1回課金し、ブロックの手数料分配・集計と一致させる。
C-10	同じ入力・前状態から全ノードが同じ状態とreceiptを生成する。外部HTTP・壁時計・乱数を合意処理で参照しない。

5. 原子性の意味

候補ブロックを作る側では、無効な新取引の一時的変更をundoし、候補から除ける必要があります。受信側では、無効な新取引を含むブロック全体を拒否します。受信した署名済みブロックから不正な部分だけを削除して採用する設計ではありません。

署名・形式検査、状態検査、資産移動、注文消費の通知順序を明示し、途中失敗を挿入して最終状態が変化しないことを試験します。observerのCommitとRollbackだけでなく、未承認候補のundoも対象です。

6. PoCの完成条件

完成には次の証拠が必要です。

1. 対象コミットと変更差分、ビルド環境、再現用コマンドを記録する。
2. ネイティブのC++プラグインがビルド・ロードできる。
3. 独立したクライアント側codecとC++側codecの固定ベクトルが一致する。
4. 各設計書の必須テストが実行され、出力と結果が保存されている。
5. 3ノードで正常取引を受け入れ、同一高さのblock hash・StateHash・ReceiptsHashが一致する。
6. 未ファイナライズの競合枝への実際の切り替えを起こし、巻き戻し・再適用後の残高と注文状態が一致する。
7. 1ノードを再起動／再同期しても、その状態と重複防止が維持される。
8. 未実施、失敗、実装済みのみ、実行成功を区別した最終報告がある。

PoC完成は、mainnetへの採用可能性や資産保管サービスとしての安全性を保証するものではありません。公開ネットワーク導入には状態移行、費用・DoS、独立レビュー、クライアント互換性など別の設計判断が必要です。

7. 実装範囲の優先順位

最初は各案の限定仕様を満たします。任意の再帰、部分約定、価格オラクル、AMM、ブリッジ、外部資産、一般公開UIは含めません。ネットワーク処理と独立したCLIで十分です。REST/Mongo/ウォレットの本格対応は、合意処理のPoCが通った後に進めます。

C++だけに閉じたバイト列の自己検証ではクライアント相互運用を確認できません。一方、単純な参照モデルだけではコンセンサスを確認できません。両者の役割を分けて証拠を残します。

Codex 共通作業指示

この文書は、IntentSettlementとAtomicBundleのどちらの作業でも最初に読みます。現在のリポジトリの AGENTS.md、ビルド規約、ユーザーから与えられた制約を確認したうえで適用してください。

1. 依頼の完了条件

設計書に基づき、公式C++ catapultへ新取引型を実装し、対象テストと隔離した私設ネットワークによる受け入れ試験を行ってください。設計説明、疑似コード、HTTPのモック、Pythonの残高モデルだけで完了しないでください。

不足する環境や権限がある場合は、実行可能な作業を進め、未実施部分・理由・必要な手順を具体的に記録します。試験結果を捏造せず、単体試験の成功をネットワーク検証の成功と表現しないでください。

2. 開始時の確認

1. README.md、docs/00-common-spec.md、対象案の設計書と指示書を読む。
2. 実装先リポジトリを確認する。存在しなければ公式 https://github.com/symbol/symbol から隔離したローカル作業コピーを用意する。
3. gitの現在のブランチ・コミット・未コミット変更と、適用される AGENTS.md を確認する。既存作業を上書きしない。
4. 基準は 6c04695555230c89a6ffa2ac4d41774dcac77ae3。異なるコミットへ実装する場合は差分とAPI変更を先に記録し、黙って基準を変えない。
5. Intentは poc/intent-settlement-v1、Atomicは poc/atomic-bundle-v1 を候補に、別作業ツリーを使う。既存名があれば衝突しない名前を選び、記録する。
6. ビルド手順・CMake構成・ネイティブテスト配置・nemgenとプラグインの読み込み方を実ソースから確認する。コマンドや既存APIを推測で作らない。

3. 作業の進め方

- M0から順に進め、各段階で具体的なファイルと検証結果を残す。通常の実装判断ごとにユーザーへ確認を要求しない。
- リポジトリ内の検索は rg を優先する。依存のない調査や読み取りを並行し、同じファイルへの編集を競合させない。
- 設計の安全性条件を緩める代替実装は禁止する。実在APIと文書案が合わない場合は、機能を保った最小変更を行い、Architecture Decision Recordに根拠を記録する。
- マルチシグやモザイク制限に対応しきれなければ、v1の範囲に従って明示的な失敗にする。黙って検査を省かない。
- 広範なリファクタリングや、関係のない依存更新を混ぜない。
- CLI引数やログへ実用秘密鍵を露出させない。fixture鍵は公開済みの使い捨てテスト専用と明記する。
- ネットワークアクセスはソース取得・依存取得と隔離テストに限定する。公開チェーンへ取引を送信しない。

4. 実装前に固定するもの

次を docs/poc/<案名>/protocol-abi.md に記録してください。配置はリポジトリ規約に合わせて変更できますが、参照を更新します。

- 取引型、版、FacilityCode、キャッシュ番号。予約値との衝突検査結果。
- 固定・可変バイト列のレイアウト、endianness、padding、署名対象、hash、全長上限。
- 合意設定、有効化条件、genesis作成条件、既存機能から変える箇所。
- 外側feeの通知順。共通仕様に従い、PoCの全実行高さでCustom処理の後にfeeが引かれる設定と実際の順序を確認する。
- 手数料、DeepCount、receipt source、署名検証コストの扱い。
- 失敗コードと、どの検査で拒否するか。

型名や関数名は「新しく作るもの」と「既存の呼び出し先」を区別します。既存のコアAPIでない名前を、実在するかのように呼び出すコードを残さないでください。

5. 検証の方針

最初に対象コンポーネントのテストを実行し、その後に関連する既存のAggregate・Transfer・HashCache・restriction・rollbackの回帰試験を行います。対象が通っただけで終了せず、設計書の各必須受け入れ項目を根拠付きで埋めます。一方、残ったリスクや必須ゲートに関係しないテストの拡大は避けます。

私設ネットワーク試験では、全ノードの同一高さを比較します。時刻が異なる任意の先端同士を比較して一致したと扱わないでください。fork試験は、ブロックを実際に作成・伝播して巻き戻しを起こします。単にobserverのRollbackを呼ぶ単体試験は、別の検証項目です。

新Merkleキャッシュを持つIntentでは、nemgenとノードに同じプラグインを読み込ませます。既存mainnetのgenesis・snapshotへキャッシュを追加する方法は、このPoCの代替になりません。

6. 残すべき成果物

各案について、少なくとも次を残してください。

成果物	内容
実装	C++ plugin、model、notifications、validators、observers、必要なcacheと設定
クライアント	注文作成・署名、取引構築、送信、結果確認の最小CLI
スキーマ	catbuffer／正規codec、署名・hash・サイズの固定ベクトル
私設ネット構成	fresh genesis、3ノード設定、起動・停止・データ初期化・再現手順
テスト	正常系、改ざん、権限、重複、fee、undo、fork、再起動
証拠	コミット、設定hash、raw payload、期待値、実測値、block/state/receipt hash、ログ
報告	実装済み／実行済み／未実施、残る課題、変更理由、再現コマンド

証拠の既定配置は artifacts/poc/<案名>/ とします。大量ログ・秘密鍵・ビルド出力を無差別にgitへ追加しません。再現に必要な小さい公開テストfixtureとサマリーを選別します。

7. 最終報告の書式

次の順で報告してください。

1. どの機能が動き、どの必須条件がまだ満たされていないか。
2. 対象コミット、作業ブランチ、主要変更ファイル。
3. 各受け入れテストIDと PASS / FAIL / NOT_RUN、証拠へのリンク。
4. 3ノード・fork・再起動の結果。実行していなければ明確に記す。
5. 設計書からの変更と理由。
6. 同じ環境で再現するための正確なコマンド。

この依頼はローカルPoCの実装と検証です。リモートへのpush、PR公開、mainnetへの配置、実資産を使った運用開始は含みません。

Codex 実装指示書 - AtomicBundle v1

目的と読込順

あなたは Symbol / catapult の C++ 実装担当である。最初に codex/00-common-instructions.md、docs/00-common-spec.md、docs/atomic-bundle-design.md を読む。現状は設計のみであり、実装・検証済みではない。

symbol/symbol@6c04695555230c89a6ffa2ac4d41774dcac77ae3 を基準に、既存署名済み AggregateComplete V3 注文を、不可分に funding / execution / sweep する DEX 限定の新しい top-level transaction pluginを実装せよ。注文 fixture は HinaBlogChannel/heika@891a2cb17a0d52c85ed85594d8283c844481a6cb の examples/intent-dex を参照する。

隔離 private chain、新規 genesis、test-only 資産だけを使う。公開ネットワーク、実鍵、mainnet 更新は対象外。基準版で動く実装を作る。

守る仕様

- 論理子操作は funding → legacy order → sweep の三つだけ。legacy は記事どおり Transfer 三つの AggregateComplete V3 一つ。
- funding / sweep は native EmbeddedTransfer V1。単独の通常 Transfer として署名しない。
- inventory が外側を署名し、P が外側 hash を連署する。maker は legacy だけ署名する。legacy P 連署は外側 P 連署の代わりにならない。
- legacy の共通検証と Aggregate 検証を全て維持する。childPlugin.publish() だけで済ませない。
- outer fee は solver が一回払う。legacy actual network fee は 0、元 MaxFee は変更しない。tip は全量回収し、funding / sweep の別 fee はない。
- P の既存 dust を保持する。sweep-all を禁止し、全モザイクの P_after=P_before を検証する。
- 消費は既存 HashCache の (legacy Deadline, canonical legacy hash) を通常経路と共有し、rollback で除去する。
- 未対応の Bonded・再帰・alias・partial・multisig・メッセージ・管理操作は明示的に拒否する。maker / inventory / P は相異なる通常の単一鍵アカウント。

M0 - 実リポジトリの確認と作業分離

適用される AGENTS.md、build / test 文書、CMake、schema generator を実読する。既存変更を確認・保護し、基準 commit から独立 worktree と poc/atomic-bundle-v1 branch を作る。dirty tree の持込みや他者の変更の reset をしない。

最低限、次の実装と対応テストを読む。

- TransactionPlugin.h、Transaction.h、EntityHasher / signature utilities。
- NotificationPublisher.cpp、AggregateTransactionPlugin.cpp、Aggregate model / validators、catbuffer の aggregate schema。
- TransactionHashObserver.cpp、UniqueTransactionHashValidator.cpp、HashCache の保存・prune・rollback。
- BatchEntityProcessor.cpp、processing / undo subscriber、UT と harvesting の検証経路。
- BlockUtils.cpp、SourceChangeObserver.cpp、native Transfer、account / mosaic restrictions、multisig 認証。
- fee debit を変える fork 設定、private-network の genesis / 起動構成。

既存 type ID と他方の PoC を検索し、非重複の実験型を記録する。架空の API・既存型番号・テスト名を作らず、新しい adapter は新設と明記する。

commit、実コマンド、実験型、設定、通知方針を plan に残す。共通の三 fork height は Height(1) 相当で genesis と全ノードに統一し、初期の通常 Aggregate V3 成功を確認する。差異は根拠付きで記録し、署名・重複検証を省いて解消しない。

M1 - wire / codec と署名対象

設計書の wire を schema / C++ model にする。body は Root32 / ActionsSize u32 / ActionCount u16 / OuterCosigCount u16 / Reserved8 / ActionStream / P cosig104、count は 3 と 1。

frame は kind u16, version u16=1, flags u16=0, reserved u16=0, index u32, payloadLength u32, payload, zeroPadTo8。kind は funding=1、legacy=2、sweep=3、index は 0/1/2。root は次の通り。

```
SHA3-256(ASCII("SYMBOL_ATOMIC_BUNDLE_ACTIONS_V1")
    || LE32(3) || LE32(byte_length(ActionStream)) || ActionStream)
```

ASCII に NUL・改行を付けない。native payload と frame の padding を区別し、異常長、余剰 bytes、非ゼロ reserved、overflow、非 canonical なモザイク順序を拒否する。

外側主署名には root までの固定 body を含め、末尾 P cosig を含めない。P の署名対象は基準版で計算する外側 tx hash。merkleSupplementaryBuffers は outer P 公開鍵 32 bytes 一つ。連署前後で hash が同じ、outer header / root 改変で署名が失敗する golden tests を作る。opaque root fixture は実行不能な codec fixture と明記し、native な実行用 fixture も別に用意する。

outer 4,096 bytes、legacy 2,048 bytes、署名 4 個、固定深さ、outer lifetime 2 時間を PoC 仮値として設定する。

M2 - C++ plugin と認証 adapter

load 可能な plugin、登録、CMake、codec、validator を追加する。新型は top-level のみ。native Aggregate の embedding 不可を一般的に緩めない。

legacy adapter は maker の AccountPublicKey、Entity、TransactionNotification、Deadline、actual fee 0 の Fee、replay-protected 主署名と、Aggregate 固有通知・連署を維持する。元 legacy bytes と hash を使い、通常 publisher による MaxFee 課金や top-level source 移動を発生させない。原本の maker signature / Deadline / MaxFee を再署名・書換えして通してはならない。

maker の Aggregate 操作権限と inventory の新型操作権限を確認する。Bundle 専用の追加制約として全五つの Transfer の signer に outgoing Transfer permission を要求する。maker は give / tip、P は get / sweep、inventory は funding。native embedded publisher だけでこの検査をしたことにしない。通常 Aggregate の既存挙動は変更しない。通常・remote・multisig アカウントの扱いも native 規則と v1 制限で固定する。

残高・制限・期限・消費状態の変更を検証し、四つの署名はそれぞれ個別に破損した負試験を作る。

M3 - 状態・fee・replay・source

同じ native cache delta 上で funding、注文、sweep を処理し、失敗時は全差分を戻す。sweep は give と tip の増分だけ。give と currency が同じなら checked addition で一つにまとめる。dust と無関係資産を動かさない。

共通仕様に従い、private chain は TotalVotingBalanceCalculationFix=Height(1) 相当の実在設定を確認し、activation height 2 以降で custom 処理後に outer fee debit が来るよう統一する。設定表記と native 通知順を実測する。funding 全量はその実行時に必要だが、outer fee のために custom 前の残高を不必要に二重要求しない。UT の MaxFee 仮徴収と、実ブロックの actual fee の違いも試験する。

legacy TransactionNotification で既存 HashCache を検証・更新する。子の fee と一緒にこの通知を省かない。同一・別ブロック、再起動、prune、rollback を試し、専用の消費 cache は増やさない。

embeddedCount=6、DeepCount=7 とする。receipt source primary は外側 index を保持し、secondary は funding 1 / legacy container 2 / legacy内部 3,4,5 / sweep 6、外側へ戻る前に 0。既存 Aggregate の相対 source 更新を adapter で閉じ込める。source trace、block limit 計数、fee receipt、receipt hash を検証する。

M4 - 三ノードの統合環境

同じ plugin の C++ catapult 三ノードを独立 genesis で起動し、public seed を無効にする。test-only keys、network ID、generation hash seed、設定 hash を記録する。

fixture / CLI で maker の事前署名と solver の構築を再現する。全ノードの hash 一致と、通常 Transfer / Aggregate の回帰を確認する。mock の残高計算で代用しない。

M5 - 攻撃対照と回帰 gate

設計書 AB-01〜18 を実施し、実行コマンドと結果を ID 別に残す。必須の難しいケースは以下。

1. 旧方式の対照：T1/T2/T3 の元ブロックに対し、早い有効 timestamp の競合ブロックへ T1 と P→攻撃者送金を入れる。private chain のみで VRF / hit / score を満たす採用条件を再現する。
2. 新方式の抽出拒否：funding embedded payload を単独 Transfer へ変換・直接収録しても、inventory の有効な独立署名がなく拒否されること。元の外側署名を流用しても通らないこと。
3. 新方式の fork：同一親から競合枝を作る。正しい Bundle 全体なら全資産移動が一緒に成立し、除外すれば全て成立しない。途中だけ残る結果がないこと。finalized 済み元ブロックの差し替えは拒否する。
4. 経路間 replay：通常→通常、通常→Bundle、Bundle→通常、Bundle A→Bundle B の四方向を、同一ブロックと別ブロックで検証する。outer の hash / signer / deadline を変えても子注文を再消費できないこと。
5. 後段失敗と再起動：test harness 限定で legacy 適用後・sweep 中・outer fee 前へ failure を注入し、funding と子消費を含む全差分が消えること。署名不正は別の前段拒否試験にし、本番検証を遅らせない。rollback 後は期限内・未消費なら正しく再署名した別 wrapper を受理する。restart 後に消費済み注文が復活しないこと。

UT 除外と強制収録した不正ブロックの拒否を別々に試す。壊れた署名・timestamp のブロックを有効な fork の代用にしない。通常 Aggregate と変更した共通経路の回帰 gate を完了する。

M6 - 納品と結果報告

実装差分、schema / codec、native fixture、三ノード設定、再実行手順、test runner、raw logs、hash / balance 比較、設計との差異、dependency / build 制約を残す。

結果は成功・失敗・環境制約で未実行・仕様未対応を区別する。コンパイル、mock、署名を省いた happy path だけで完了にしない。阻害されても可能な作業は続け、未検証項目を明示する。

完了時は仕様、型・隔離条件、抽出防止・replay・fee・source・rollback の証拠と制約を報告する。mainnet 適用や監査完了を主張しない。

一次資料

コードは 設計書の一次資料 を参照。公式 Plugin 説明 は構成の参考であり、固定版コードの代わりにしない。